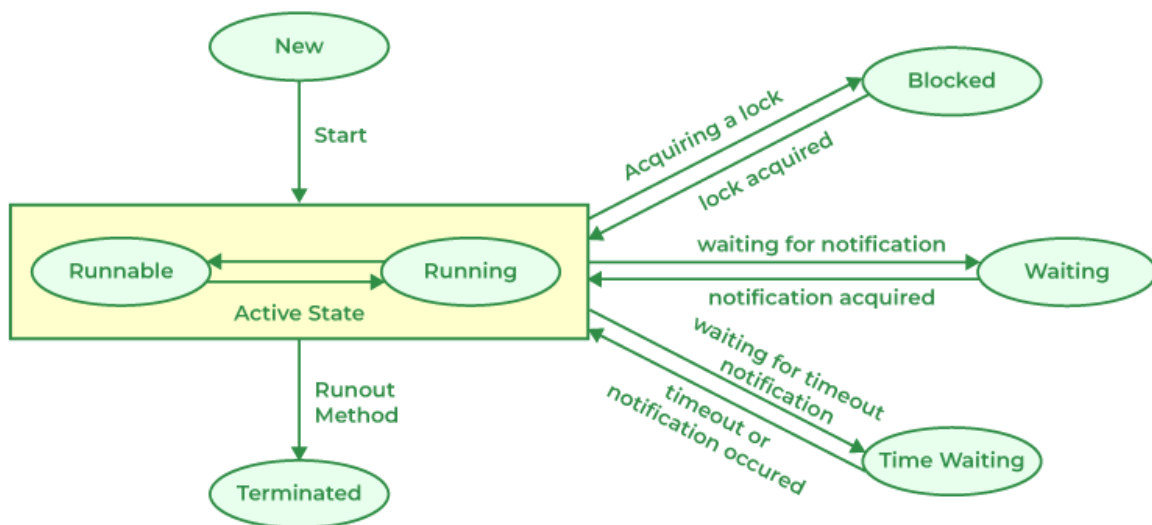


Lifecycle and States of a Thread in Java

Last Updated : 31 Mar, 2026

The lifecycle of a thread in Java defines the various states a thread goes through from its creation to termination. Understanding these states helps in managing thread behavior and synchronization in multithreaded applications.

- A thread passes through multiple states during execution
- The thread scheduler controls state transitions
- Helps in efficient thread management and debugging



There are multiple states of the thread in a lifecycle as mentioned below

1. New

A thread is in the new state when it is created but has not yet started execution, so its code has not begun running.

```
public static final Thread.State NEW
```

2. Runnable

Thread state for a runnable thread. A thread in the runnable state is executing in the Java virtual machine but it may be waiting for other resources from the operating system such as a processor.

public static final Thread.State RUNNABLE

3. Blocked

A thread is in the blocked state when it is waiting to acquire a lock that is currently held by another thread.

public static final Thread.State BLOCKED

4. Waiting

Thread state for a waiting thread. A thread is in the waiting state due to calling one of the following methods:

- `Object.wait` with no timeout
- [`Thread.join`](#) with no timeout
- `LockSupport.park`

public static final Thread.State WAITING

5. Timed Waiting

Thread state for a waiting thread with a specified waiting time. A thread is in the timed waiting state due to calling one of the following methods with a specified positive waiting time:

- `Thread.sleep`
- `Object.wait` with timeout
- `Thread.join` with timeout
- `LockSupport.parkNanos`
- `LockSupport.parkUntil`

public static final Thread.State TIMED_WAITING

6. Terminated

Thread state for a terminated thread. The thread has completed execution.

public static final Thread.State TERMINATED